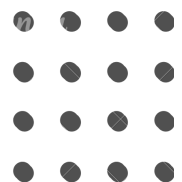




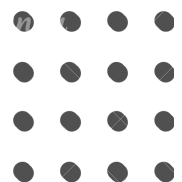
VMES Grinds





Pilot Program - Week 5 - Day 2

Stack





Leetcode: 1475
Final Prices With Special Discount





1475. Final Prices With a Special Discount in a Shop

Solved

Easy

Topics

Companies

Hint

You are given an integer array `prices` where `prices[i]` is the price of the i^{th} item in a shop.

There is a special discount for items in the shop. If you buy the i^{th} item, then you will receive a discount equivalent to `prices[j]` where `j` is the minimum index such that `j > i` and `prices[j] <= prices[i]`. Otherwise, you will not receive any discount at all.

Return an integer array `answer` where `answer[i]` is the final price you will pay for the i^{th} item of the shop, considering the special discount.





Example 1:

Input: prices = [8,4,6,2,3]

Output: [4,2,4,2,3]

Explanation:

For item 0 with price[0]=8 you will receive a discount equivalent to prices[1]=4, therefore, the final price you will pay is $8 - 4 = 4$.

For item 1 with price[1]=4 you will receive a discount equivalent to prices[3]=2, therefore, the final price you will pay is $4 - 2 = 2$.

For item 2 with price[2]=6 you will receive a discount equivalent to prices[3]=2, therefore, the final price you will pay is $6 - 2 = 4$.

For items 3 and 4 you will not receive any discount at all.

Example 2:

Input: prices = [1,2,3,4,5]

Output: [1,2,3,4,5]

Explanation: In this case, for all items, you will not receive any discount at all.

Example 3:

Input: prices = [10,1,1,6]

Output: [9,0,1,6]



Problem Explanation Thought Process





Initial Thought Process

To compute each item's final price:

- Naive approach: check every future item until finding a valid discount.
- But this is repeated work for every index.
- Better: use a monotonic stack to track candidates and assign discounts efficiently.



Algorithm Evolution Overview





Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Check every pair (i, j)
Monotonic Stack	$O(n)$	$O(n)$	Efficient next-smaller search



Brute Force





Idea:

- For each index i , scan right until you find the first j such that $\text{prices}[j] \leq \text{prices}[i]$.
- Apply discount immediately.

Why It Works

- It directly follows the rule: find first $j > i$ that is $\leq$.
- Simple to implement.
- But each element may scan many positions $\rightarrow$ quadratic time.



```
class Solution(object):
    def finalPrices(self, prices):
        """
        :type prices: List[int]
        :rtype: List[int]
        """
        n = len(prices)
        if n <= 1:
            return prices
        for i in range(n-1):
            for j in range(i+1,n):
                if prices[i] >= prices[j]:
                    prices[i] -= prices[j]
                    break
        return prices
```



Time Complexity (Brute Force)

- Time: $O(n^2)$ worst case.
- Space: $O(1)$.



Monotonic Stack





Idea:

- Use a stack to store indices of prices waiting for discount.
- As we traverse:
 - While current price is $\leq$ price at stack top, we found the discount for that index.
 - Assign discount immediately.
- Push current index.

Why It Works

- Stack ensures each index is processed once.
- Maintains a decreasing sequence $\rightarrow$ next time you find a smaller value, it is the correct discount.
- Eliminates redundant scanning.



```
class Solution(object):
    def finalPrices(self, prices):
        """
        :type prices: List[int]
        :rtype: List[int]
        """
        n = len(prices)
        ans = prices
        stack = []
        for i in range(n):
            while stack and prices[stack[-1]] >=
prices[i]:
                Fi= stack.pop( )
                ans[Fi] = prices[Fi] - prices[i]
                stack.append(i)
        return ans
```




Time Complexity (Stack)

- Time: $O(n)$ — each index pushed + popped at most once.
- Space: $O(n)$ for stack.



Extra Study Material –Visualizations–





Complexity Comparison



Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Simple but slow; checks every pair (i,j)
Monotonic Stack	$O(n)$	$O(n)$	Efficient next-smaller-to-the-right finder



Key Takeaway





This problem is a next-smaller-to-the-right pattern:

- Brute force scans repeatedly.
- Stack remembers candidates efficiently.
- Every element finds the first smaller-or-equal value in one pass.



Thank You !

